

TINY VLM

RLS Entrance Challenge | Rapport technique | Niveau 1 | 27 septembre 2026

1. Probleme

Construire un modele qui observe une scene synthetique 64 x 64 et genere sa description en un mot, caractere par caractere. La question centrale est de distinguer la lecture de l'image de la simple prediction de l'orthographe. Les splits officiels ShapeScenes sont conserves : 20 000 / 2 000 / 2 000 / 2 000 exemples, seed 42.

2. Architecture et choix

Image RGB	(B, 3, 64, 64)
CNN : 3 Conv stride 2 + ReLU, puis AvgPool	(B, 128, 4, 4)
Flatten spatial + Linear adapter	(B, 16, 128)
Image + embeddings des lettres + positions	(B, 16 + T, 128)
2 blocs : attention 4 tetes + MLP	(B, 16 + T, 128)
Dernier visuel + lettres : LayerNorm + Linear	(B, T + 1, 27)

Figure 1. Dimensions du pipeline. T est le nombre de caracteres fournis ; la premiere des T+1 sorties vient du dernier token visuel.

521 307 parametres, d_model=128, quatre tetes de 32 dimensions, MLP 128-512-128. Une grille 4 x 4 conserve plusieurs positions et limite le cout quadratique. Cette compacite sacrifie du detail ; aucune ablation ne prouve que ce choix est optimal. Le modele est plus petit que l'ordre de grandeur indicatif du brief.

3. Implementation et correction

Attention manuelle : $\text{softmax}(\mathbf{QK}^T / \sqrt{32} + \text{masque})\mathbf{V}$, puis fusion des tetes et projection. Les 16 tokens visuels se voient tous, sans voir le texte. Chaque lettre voit l'image et son prefixe. Les positions apprises encodent l'ordre ; LayerNorm avant chaque sous-bloc et connexions residuelles facilitent l'optimisation.

Vocabulaire : 26 lettres + eos. Le dernier token visuel predit la premiere lettre, sans bos. Teacher forcing : une entree r predit e, e predit d, d predit eos. Le padding des cibles vaut -100 et est ignore par CrossEntropyLoss. Les entrees completees valent 0, a droite ; elles ne peuvent influencer les positions utiles passees.

108 tests passent : reference PyTorch de l'attention (tolerance usuelle 1e-5 en float32), causalite, gradients, alignement premiere lettre, padding, parser, arret eos et accord CPU/GPU. Les composants interdits ne sont pas utilises dans le modele.

4. Configuration d'entrainement

Element	Valeur
Donnees	Generateur RLS inchange ; uint8 en stockage, float32 / 255 au chargement
Architecture	CNN 32/64/128 canaux, grille 4 x 4 ; 2 blocs, d=128, 4 tetes
Optimisation	AdamW, learning rate 0,001 ; weight decay 0,01 ; clipping norme 1
Budget	Batch 64 ; 10 epochs ; seed 42 ; float32 ; aucun preentrainement
Selection	Checkpoint de perte validation minimale ; test jamais utilise pour choisir
Materiel	NVIDIA GeForce GTX 1650 ; Intel(R) Core(TM) i5-10500H CPU @ 2.50GHz ; 4 threads CPU ; Windows-11-10.0.26200-SP0
Logiciel	Python 3.12.14 ; PyTorch 2.8.0+cu126 ; CUDA 12.6

5. Experiences et protocole

E0. Hypothese avant execution : memoriser un batch de 16 exemples jusqu'a une loss inferieure a 0,02. Resultat apres 600 pas : loss 0.000372 nat/token et 100.00 % de mots corrects sur le lot. Cela valide la capacite a apprendre ce lot, sans mesurer la generalisation.

E1. Hypothese avant execution : les images reelles ameliorent exact-match et attributs par rapport aux pixels nuls. Seule variable modifiee : images mises a zero pendant entrainement et evaluation. Initialisation, ordre des lots, architecture, epochs, optimiseur et splits sont identiques. Une seed par condition : dispersion entre seeds non estimable.

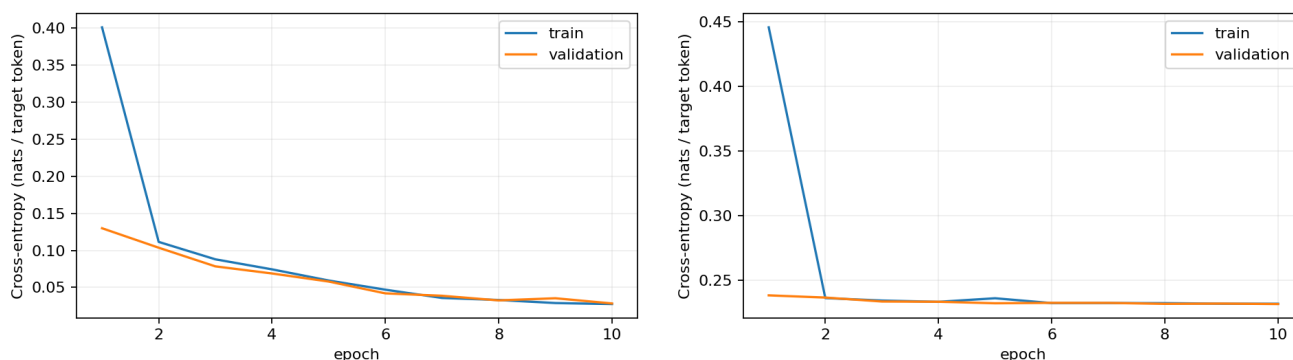


Figure 2. Loss train/validation en nats par token sur NVIDIA GeForce GTX 1650. A gauche, modele visuel ; a droite, aveugle. Le plateau aveugle illustre l'information manquante sur la scene.

La perte est ponderee par le nombre de tokens non ignores. Les CSV conservent chaque epoch ; les checkpoints sauvegardent poids, optimiseur et historique. La reprise repart de la derniere epoch terminee, avec l'ordre des batches determine par seed + epoch.

Le cours, les tests supplementaires et l'implementation ont ete prepares avec Codex, comme declare dans AI_USAGE.md. Le generateur et les tests officiels sont conserves. La comprehension du candidat doit etre demontree personnellement a l'entretien.

6. Resultats mesures

Mesure	Visuel test	Aveugle test	Visuel heldout
Exact-match	68.05 %	1.70 %	0.00 %
Taille	84.80 %	33.28 %	80.14 %
Couleur	74.83 %	20.10 %	75.80 %
Forme	73.81 %	15.30 %	23.32 %
Relation	44.22 %	0.00 %	44.63 %
Nombre d'objets	99.90 %	49.00 %	97.60 %
Lettres libres	68.99 %	16.17 %	47.29 %
Lettres, prefixe vrai	98.52 %	87.34 %	89.73 %

Tableau 1. Predictions des checkpoints retenus sur NVIDIA GeForce GTX 1650, n=1 seed/condition. Pas d'ecart-type entre seeds estime. Test : 2 000 images, 3 020 objets, 1 020 relations ; heldout : 2 000 images, 2 950 objets, 950 relations.

Les attributs sont compares par emplacement, sans autocorrection : `largerredcirle` conserve taille et couleur, mais sa forme est fausse. Taille/couleur/forme sont agregees sur les objets de reference ; relation sur les scenes a deux objets. Le nombre d'objets est mesure separement.

Les lettres libres sont comparees position par position, avec la longueur maximale au denominateur. La mesure avec prefixe vrai exclut eos et padding et utilise les bonnes lettres precedentes. Elle ne represente pas un decodage autonome.

7. Analyse et incertitude

Le modele visuel atteint 68.05 % d'exact-match contre 1.70 % pour E1. Ce contraste soutient l'utilisation de l'image dans ce run. Le baseline aveugle obtient pourtant 87.34 % des lettres avec prefixe vrai : une grande part de la sequence s'explique par l'orthographe, sans vision.

L'exact-match est de 99.49 % sur 980 scenes a un objet et 37.84 % sur 1020 scenes a deux objets. Parmi 639 erreurs strictes, 381 sont exactement equivalentes apres inversion des objets et de la relation. Exemple : A below B devient B above A. Le generateur ne fixe pas un ordre canonique ; le score officiel reste inchange.

Les 258 autres erreurs montrent que l'ambiguite d'ordre n'explique pas tout. Les scenes a deux objets et les relations restent difficiles. Les confusions ordonnees de formes peuvent aussi inclure des permutations d'objets ; elles ne prouvent pas a elles seules une faiblesse du CNN.

Heldout : aucun mot exact. Taille et couleur restent assez souvent reconnues, mais la forme chute a 23.32 %. Exemple : `largeyellowcross` devient `largeyellowcircle`. Hypothese exploratoire : associations couleur-forme apprises sans recombinaison fiable. Aucune intervention causale ni repetition sur trois seeds n'a ete faite ; niveau 2 non revendique.

8. Systeme : benchmark S1

Batch	CPU, images/s	GPU, images/s
1	61.88 +/- 7.62	98.47 +/- 5.52
16	422.40 +/- 30.51	1434.32 +/- 116.73
64	507.68 +/- 24.40	4414.42 +/- 9.09
256	495.07 +/- 40.45	4455.75 +/- 7.36

Tableau 2. Moyenne +/- ecart-type echantillon sur 5 repetitions de 10 pas, apres 5 pas de chauffe. CPU i5-10500H (4 threads), GPU GTX 1650. Float32, 16 tokens visuels + 45 caracteres. Donnees brutes : benchmarks/S1_throughput/raw.csv.

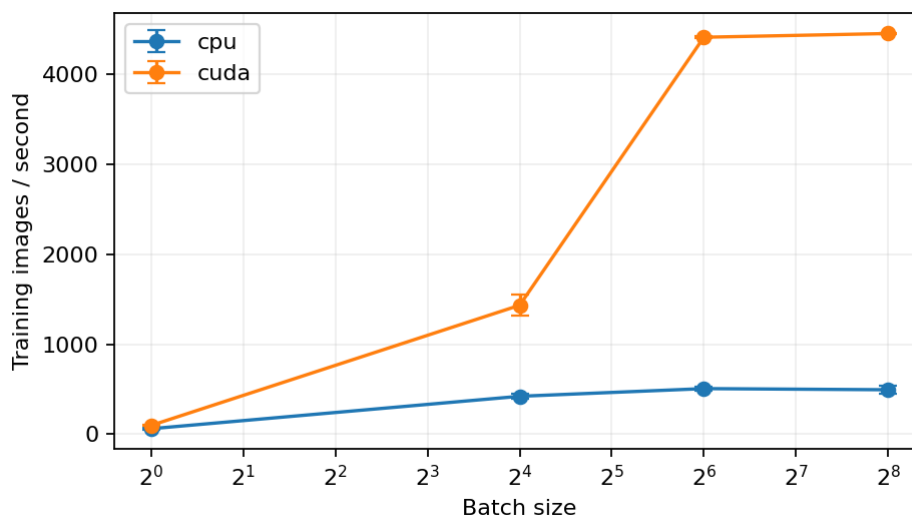


Figure 3. Le GPU gagne pour les quatre batches mesures, sans croisement observe. Le gain est moindre a batch 1 et le debit GPU plafonne entre 64 et 256 dans ce protocole.

Les tenseurs sont synthetiques et deja sur le peripherique. Le timer inclut forward, loss, backward, clipping et AdamW ; il exclut DataLoader, transferts et ecritures. La chauffe exclut les initialisations ; synchronize avant et apres le timer attend le travail CUDA asynchrone. Les essais S1 sont separes des entrainements.

L'augmentation du batch amortit les couts fixes et offre plus de parallelisme. Sans profiler, le plateau ne suffit pas a identifier un goulot precis. Une matrice d'attention a $B \times H \times T^2$ elements ; a $B=64$, $H=4$, $T=61$, elle occupe environ 3,63 Mio en float32, hors autres activations et etats d'optimiseur.

9. Limites et suite

Une seule seed, scenes synthetiques, une architecture, parser strict et ordre des objets ambigu. Pas de profiling ni d'etude memoire de niveau 2. Prochaine experience : comparer grille 4 x 4 et un seul vecteur visuel, en fixant le reste et en repetant sur trois seeds. Verifier aussi la recombinaison heldout. L'assistance IA est declaree ; l'aisance orale reste a travailler.

References

RLS, Project Brief et template c21f9da : github.com/RLS-ResearchLab/RLS-Entrance-Challenge.

Vaswani et al., Attention Is All You Need, 2017 : arxiv.org/abs/1706.03762.

PyTorch 2.8, scaled_dot_product_attention et CrossEntropyLoss : docs.pytorch.org/docs/2.8/ (liens precis dans README).